

OPERATING SYSTEMS (UE24CS242B)

Unit 2 – IPC, Threads and Concurrency

Lecture 5 - Multithreading Models, Thread Creation & Scheduling

Multithreading Models

Multithreading models describe how **user-level threads (ULTs)** are mapped to **kernel-level threads (KLTs)**. Since user threads are managed by thread libraries and kernel threads are managed by the operating system, a mapping mechanism is required to allow user threads to execute on physical CPUs.

Different operating systems adopt different mapping models depending on performance, scalability, and implementation complexity.

The three primary multithreading models are:

- Many-to-One
- One-to-One
- Many-to-Many

Many-to-One Model

In the many-to-one model, multiple user-level threads are mapped to a **single kernel thread**. Thread management is entirely handled in user space by thread libraries, and the kernel is unaware of multiple threads within the process.

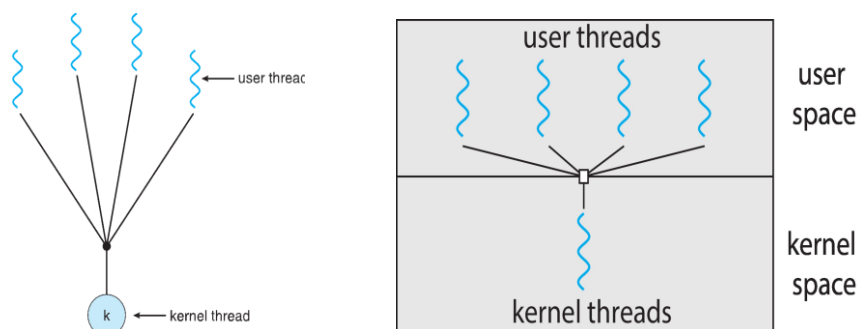
Because all user threads map to one kernel thread, only one thread can execute in kernel mode at any given time. If one thread performs a blocking system call, the entire process blocks. This significantly reduces concurrency.

Additionally, since only one kernel thread exists, threads cannot run in parallel on multicore systems. Even if multiple CPUs are available, execution remains serial.

Due to these limitations, very few modern systems use this model today.

Examples:

- Solaris Green Threads
- GNU Portable Threads



One-to-One Model

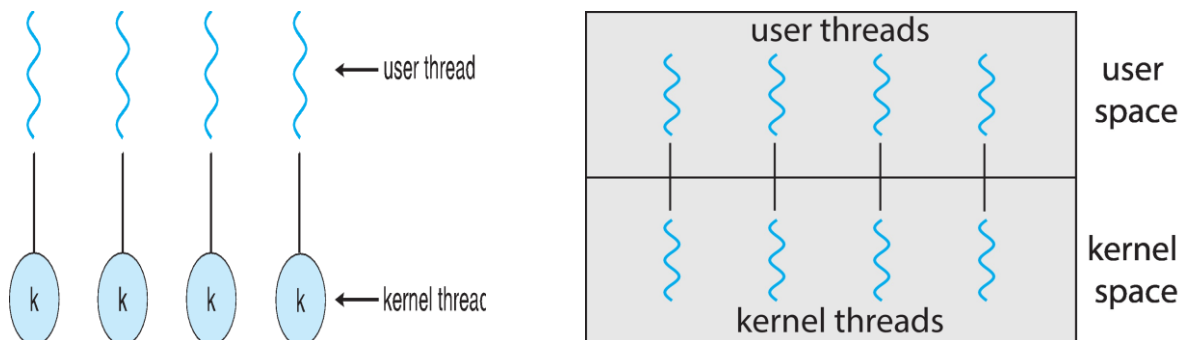
In the one-to-one model, each user-level thread maps to a **separate kernel thread**. Whenever a user thread is created, a corresponding kernel thread is also created.

This allows true concurrency because multiple threads can execute in parallel on multicore systems. Blocking of one thread does not affect others, since each has its own kernel thread.

However, creating kernel threads involves overhead. Therefore, operating systems often limit the number of threads per process to avoid excessive resource consumption.

Examples:

- Windows
- Linux
- Solaris 9 and later



Many-to-Many Model

The many-to-many model multiplexes many user threads over many kernel threads. This model combines the benefits of the previous two models.

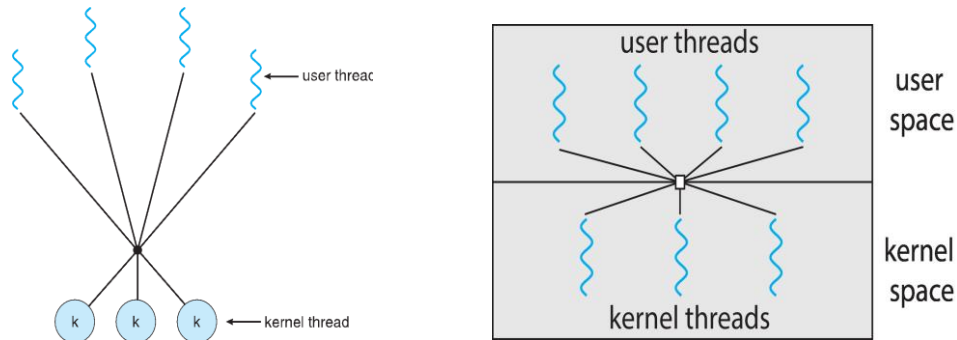
Here, the operating system creates a sufficient number of kernel threads, and the thread library schedules user threads onto these kernel threads. This allows:

- High concurrency
- Efficient resource usage
- Parallel execution on multicore systems
- No blocking of entire process when one thread blocks

This model is more flexible and scalable than the earlier models.

Examples:

- Solaris (prior to version 9)
- Windows with ThreadFiber package



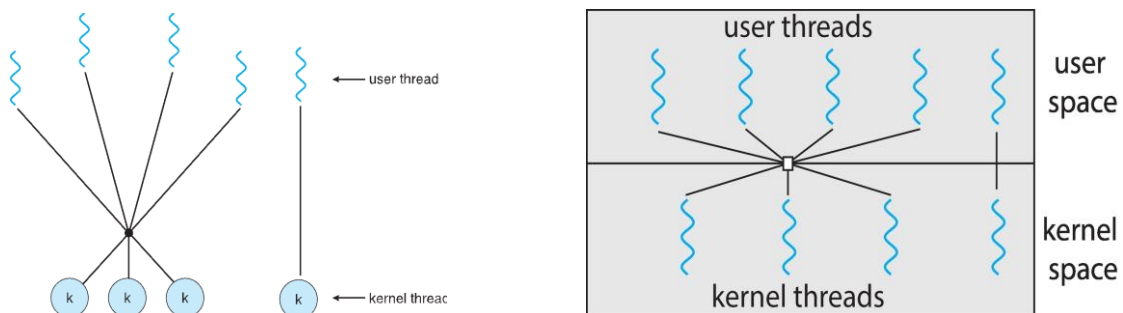
Two-Level Model

The two-level model is similar to the many-to-many model but adds one enhancement: it allows a user thread to be **bound to a specific kernel thread**.

This binding is useful when a thread requires guaranteed CPU access or real-time responsiveness. It combines the flexibility of many-to-many mapping with the control of one-to-one mapping.

Examples:

- IRIX
- HP-UX
- Tru64 UNIX
- Solaris 8 and earlier



Pthreads (POSIX Threads)

Pthreads is a **POSIX standard API** defined under IEEE 1003.1c. It provides a standardized interface for thread creation, management, and synchronization.

It is important to note that Pthreads defines only the **specification**, not the implementation. This means different operating systems can implement Pthreads either at the user level or kernel level while still conforming to the same API.

Pthreads is widely used in UNIX and UNIX-like systems such as:

- Linux
- Solaris

- macOS

Thread Creation Using Pthreads

Threads are created using the `pthread_create()` function. This function requires:

- Thread identifier
- Thread attributes
- Function to execute
- Parameters to pass to the function

Once created, each thread begins execution in the specified function.

Thread Joining

Thread joining is performed using `pthread_join()`. This allows one thread to wait for the completion of another thread before proceeding. It is commonly used to ensure all threads complete execution before program termination.

Thread Scheduling

When threads are supported, operating systems schedule **threads rather than processes**. Thread scheduling determines how threads are mapped to CPU time.

There is an important distinction between:

- User-level threads
- Kernel-level threads

In many-to-one and many-to-many models, thread libraries schedule user threads onto **Lightweight Processes (LWPs)**, which act as intermediaries between user threads and kernel threads.

Lightweight Processes (LWP)

A Lightweight Process is a kernel-supported entity that provides the necessary context for user threads to access CPU resources. LWPs act as a bridge between user-level thread management and kernel scheduling.

The thread library maps user threads to LWPs, and the kernel schedules LWPs onto physical CPUs.

Thread Scheduling Scopes

Thread scheduling operates under two contention scopes:

- **Process-Contention Scope (PCS)**

In PCS, scheduling competition occurs **among threads within the same process**. The thread library decides which user thread runs on the available LWP.

This type of scheduling is managed at the user level and is common in many-to-one and many-to-many models.

- **System-Contention Scope (SCS)**

In SCS, scheduling competition occurs **among all threads in the system**. Kernel threads compete globally for CPU time, and scheduling is performed by the OS scheduler.

This model provides broader fairness and is used in one-to-one threading systems.

Pthread Scheduling

The Pthread API allows programmers to specify the scheduling scope during thread creation.

Two options exist:

- PTHREAD_SCOPE_PROCESS → uses PCS scheduling
- PTHREAD_SCOPE_SYSTEM → uses SCS scheduling

However, not all operating systems support both scopes. For example:

- Linux and macOS allow only PTHREAD_SCOPE_SYSTEM.

Pthread Scheduling API

The scheduling scope is set using thread attributes. The program:

1. Initializes thread attributes
2. Retrieves current scheduling scope
3. Sets desired scope
4. Creates threads using those attributes
5. Joins threads after execution